

Reflective Skill Optimisation

Proposal, research lineage, and implementation schema for a no-RL self-improvement loop over the diagnostic channel

AgentIA Engineering

2026-09-29

What this document is. A proposal for making the AgentIA coding agent more reliable using **reflective, no-RL optimisation**: a language model reflects over recorded evidence and proposes edits to skill documents, which are admitted only on measured contribution and evicted when they fall below an evidence floor. **No weight updates, no training, no GPUs** — the platform has none, and the evidence says a reflective curator captures a measurable fraction of what a trained one achieves.

It is written *after* the first real run (`agentia_first_baseline.tex`) and after the instrument shipped, so it separates what has been measured from what is merely proposed. Where a claim is not yet measured, it is labelled as such.

Contents

1	The proposal in one page	2
2	Why the diagnostic channel precedes the loop	2
3	The three conditions the evidence requires	3
3.1	Condition 1 — Per-skill contribution, by leave-one-out	3
3.2	Condition 2 — Eviction with an evidence floor	3
3.3	Condition 3 — Diagnostics richer than pass/fail	4
4	Research lineage	4
4.1	Reflective text-space optimisation (no weights)	4
4.2	A trained curator, and the fraction reflection recovers	4
4.3	Null results, which are part of the lineage	4
4.4	Where the levers rank	5
5	Implementation schema	5
5.1	The channel, end to end	5
5.2	What exists, and what is deferred	6
6	What is measured today	6
6.1	The first real run	6
6.2	The artifact question, re-measured	6
7	The gate on building the loop	7
7.1	So the decision is: grow the corpus, or do not build the loop	7
8	Honest limits	8
9	Decision record	9

1 The proposal in one page

Element	Proposal
Objective	Conformance , measured by the deterministic rule instrument — not build success, which is a binary whole-corpus outcome that no skill edit can move detectably at this sample size.
Proposal mechanism	A reflective LLM proposes a bounded edit set against skill documents. No gradients, no fine-tuning, no RL.
Admission	Measured contribution, by leave-one-out: compare outcomes with the skill present against outcomes with it absent.
Eviction	Remove a skill only when its measured contribution is below a stated floor . “No proven benefit” is not grounds for removal.
Guardrail	The existing strict-improvement gate is demoted from acceptance criterion to regression veto. It keeps its job; it loses its vote.
Bound	A round considers a bounded number of skills, because attribution cost is (multiplicative) skills $\times$ tasks.
Gate to build	The corpus report must first show the conformance measure <i>varies</i> across real sessions. If every session scores identically, there is nothing to move.

The loop, in the shape it would take:

```

session runs
|
v
compliance gate evaluates each stage's candidate
|
| (pass)          \ (fail) -> rule_id, severity, artifact, stage
v                  v
artifacts persisted    DIAGNOSTIC CHANNEL          <-- exists (feature 015)
                        |
                        v
                    aggregate by rule across sessions
                    (which rules recur, where, how often)
                        |
                        v
                reflective LLM proposes skill edits
                targeting the RECURRING rules
                        |
                        v
                candidate skill -> fresh sessions -> measure
                        |
                        v
                admit only on measured contribution;
                evict skills below the evidence floor

```

2 Why the diagnostic channel precedes the loop

This ordering is the proposal’s load-bearing decision, and it is not a preference. It follows from the single largest effect in the evidence base.

CoEvoSkills (arXiv:2604.01687) reports **+40.5 points** on SkillsBench with a co-evolving diagnostic verifier. Its own ablation:

CoEvoSkills variant	Score
Full (co-evolving diagnostic verifier)	71.1
Opaque pass/fail oracle only	41.1 (−30.0)
No evolution at all	42.4 (−28.7)

A loop fed only a binary verdict performs no better than no loop at all. 41.1 with a pass/fail oracle against 42.4 with nothing: the loop is not merely weaker without structure, it is *indistinguishable from absent*. The diagnostic channel is therefore worth more than the loop, and building the loop first would have produced a component that cannot be shown to do anything.

Three further results point the same way:

- An LLM-as-judge is the instrument the literature most distrusts, and it fails concretely: in the direct SkillOpt-on-code evaluation ([arXiv:2609.12742](#)), a candidate that wrote **an empty file** scored *higher* on the judge than the baseline. A scoring channel that rewards emptiness cannot steer anything.
- Hand-written rule quality is largely content-independent — random rules and curated rules scored **63.8%** each in the guardrails study ([arXiv:2604.11088v2](#), 679 files, 25,532 rules) — which is an argument for a *deterministic* instrument rather than for cleverer prose.
- Rules about architecture measured **worst** of the rule types (53.4% against 63.8% for tool/process rules). Since AgentIA’s whole subject is layered architecture, the signal must come from the platform’s own gate, not from a rule recipe imported from elsewhere.

This is why feature 015 shipped the instrument and *deferred* the optimiser. The decision is recorded in `specs/015-diagnostic-skill-evolution/spec.md` under “Deferred: the optimizer”.

3 The three conditions the evidence requires

3.1 Condition 1 — Per-skill contribution, by leave-one-out

A whole-corpus pass-rate comparison cannot attribute an effect to a rule: with one document, co-applied edits confound one another, and “the output got better” is not “this rule was responsible”. The primitive that resolves this is training-free and computes

$$\text{delta}(\text{task}, \text{skill}) = \text{r}(\text{task}, \text{skill}) - \text{r}(\text{task}, \text{absent})$$

by replaying with the skill removed. This is decision **D3** in `research.md`. Its cost is multiplicative — skills $\times$ tasks — which is why the per-round skill bound (D10) is not a convenience but the thing that keeps measurement affordable.

3.2 Condition 2 — Eviction with an evidence floor

A skill library that can only grow degrades until injecting a skill is worse than injecting nothing ([arXiv:2605.22148](#)). The repair is contribution-based eviction, and the floor is what stops measurement noise from driving removals: “we could not prove it helps” and “it hurts” are different findings.

The direction is corroborated by operation ablations ([arXiv:2605.29440](#)):

Operation set	Score
Add only	47.0
Add + remove	48.3
Add + rewrite	53.5
All three	59.0

Add-only editing — feature 014’s behaviour — is the worst configuration. This is decision **D4**.

3.3 Condition 3 — Diagnostics richer than pass/fail

The channel must carry *structure*: which rule, on which artifact, in which stage, at which severity. A score says a session went badly; it does not say what to change. `rule_histogram` is the part a curator can act on, and per-stage attribution converts a session total into a profile. This is decision **D7**; the implementation reads the verdicts the platform *already* computes rather than re-evaluating anything.

The honest limit of this condition. Per-stage attribution adds *resolution*, not *evidence*. Re-checking the same task repeatedly raises the session count but not the number of distinct tasks, and the channel is a fixed hand-written rule set: a defect class nobody encoded reads clean. It measures **conformance**, **not correctness** — a well-layered but broken service scores full marks.

4 Research lineage

Three lineages, in ascending order of how directly they bear on this proposal.

4.1 Reflective text-space optimisation (no weights)

GEPA (arXiv:2507.19457) reports **+6% average, up to +20%** over GRPO with **up to 35× fewer rollouts**. SkillOpt (arXiv:2605.23904) reports best-or-tied in **52 of 52** cells, **+5.4** over its strongest per-cell competitor (82.3 vs 76.9), across seven models at roughly **+17.6** average. Both work on text, not weights.

4.2 A trained curator, and the fraction reflection recovers

CODESKILL (arXiv:2605.25430) is the closest controlled comparison, and it is the source of the number this proposal leans on:

Curator	Score	
No skill management	29.32	
Prompt-based skill management (GPT-5.4-mini)	35.25	(+5.93, 54% of headline)
SFT curator	34.54	(+5.22, 47% of headline)
CODESKILL (RL curator)	40.35	(+11.03 headline)

A prompt-based curator with no training, no iteration and no verifier feedback captures roughly half the gain of the trained RL curator. That is the measured floor reflection starts from — not zero. The RL half (+5.1 to +5.8) is the part this platform cannot reach, and it is honest to say so: the trained run cost approximately **920 H100-GPU-hours**.

4.3 Null results, which are part of the lineage

- The direct test of SkillOpt on repository-level code (arXiv:2609.12742) reports GEPA docs **+4.9 pp** and SkillOpt **+0.1 pp**, *neither significant*, over 20–26 held-out tasks per repo.
- Gloaguen et al. (arXiv:2602.11988) found **no general gain** and **+20% cost**; the v2 figure is “7%” with a “does not generally improve” verdict.
- LLM-written skills measured **+0.0 pp** against **+16.2 pp** for human-written ones in the Ratchet study.

The reports themselves disagree on how to frame these; the fact sheet records that report 1 calls the repo-level question untested while report 2 reports the direct null. For this proposal the reading is simple: **the effect is real in text-space but bounded, and it is procedural — consistency, not capability.**

4.4 Where the levers rank

The two reports rank the available levers, and they differ in a way this proposal must respect:

	Report 1 (capability)	Report 2 (zero-compute EV)
Best	model capability	verification-guided best-of- k
then	harness / interface design	executable house standard
then	verification-guided selection	conformance score as metric
then	RL post-training	harness / pipeline
then	RL-trained skill banks	retrieval
then	optimised text skills	bounded self-repair
worst	hand-written rules	skill-document optimisation

On capability, text skills rank above hand-written rules. On expected value per unit of compute, skill-document optimisation ranks **last**. This proposal is therefore explicitly the *last* lever, and it says so rather than implying otherwise. It is being built because the mechanism is cheap *once the evidence exists* — and because the evidence is the thing the platform just learned it did not have.

5 Implementation schema

5.1 The channel, end to end

```

stage candidate
|
v
compliance gate (security_service + test_analysis_service, merged)
|  LOCAL violation  -> reject, consume a correction attempt
|  ACCUMULATED      -> record, never reject
v
accepted artifact set --> persisted to the workspace
|
v
session terminal -----> diagnose(artifacts) [instrument]
|
|                                +--> score, raw_penalty, density
|                                +--> rule_histogram, findings[]
|                                +--> evaluable / not evaluable
|                                +--> unverified (synthetic build)
v
stage_attribution(journal)
|  reads initial_verdict per stage  (introduction, not persistence)
|  per stage: outcome, requests, rule_histogram, passed|None
v
session_diagnostic_records  (one row per session)
|
v
corpus_report: population, exclusions+reasons, conformance distribution,
               correction effort, distinct tasks, cost if data exists
|
v
[DEFERRED] skill_contribution: leave-one-out delta per skill

```

```

      |
      v
[DEFERRED] reflect -> candidate skill -> fresh sessions -> admit/evict

```

5.2 What exists, and what is deferred

Component	Where	State
Deterministic instrument	services/conformance_diagnostics.py	built
Per-stage attribution	same, reading the stage journal	built
Session record (durable)	models/diagnostics.py	built
Corpus report	services/corpus_report.py	built
Batch driver	scripts/run_corpus_baseline.py	built
Skill generator (reflection)	scripts/skillopt/reflect.py	exists, fed the wrong signal
Per-skill contribution	services/skill_contribution.py	deferred
Eviction with a floor	scripts/skillopt/currency.py	deferred

The reflector currently receives a derived build exit code, a terminal status and artifact paths — the “opaque verdict” of §2. It should receive the rule histogram: which rules fired, how often, at which stage. The channel exists; the wiring does not. That is a small change to a built component, and it is deferred only because the channel’s variance across real sessions is still unmeasured.

6 What is measured today

6.1 The first real run

From `agentia_first_baseline.tex`: one session, blueprint minimal, provider `deepseek`, model `deepseek-flash`.

Measure	Value
Model calls	15
Input tokens	26,351
Output tokens	139,679
Cost	\$0.085295889 (15 of 15 calls priced)
Terminal state	BLOCKED

Every stage consumed its entire correction budget. The first four were accepted on their third attempt; TEST never converged. This is *systematic procedural non-compliance* — exactly the failure mode a reflective loop targets, and precisely the signal the compliance gate already computes.

6.2 The artifact question, re-measured

The baseline document recorded a diagnostic record with `artifact_count = 0`, `evaluable = false` on a session whose first four stages had persisted artifacts, and named that the critical path. That has now been traced to its end, and the conclusion is different from the one the record implied.

Question	Answer, and how it was established
Are artifacts lost on the stage-runner exhaustion path?	No. A scripted model reproducing the baseline’s exact pattern (two rejections then an acceptance for each of four stages, three rejections for TEST) returns all seven persisted artifacts in the final state — through the stage loop, through the compiled graph, and through the production <code>stream+update</code> accumulator.
Why did the record read zero?	It was fabricated test output . The single stored record held five EXHAUSTED stages with empty rule histograms — the shape the test fixtures wrote under the production session key, which is the collision commit <code>dbcfa2e</code> fixed.
Is a real blocked session diagnosable now?	Yes. The reproduction records <code>artifact_count = 7</code> , <code>evaluable = true</code> , with the four corrected stages carrying the rules that rejected their first candidate.

A regression test now pins each of those three claims (`test_diagnostic_record.py`), so the “not yet reproduced” note in `dbcfa2e` is closed.

One genuine defect *was* found by tracing this, and it matters more than the one that was assumed. `stage_attribution` read `verdict.get("passed", not violations)` on an absent verdict, so a stage whose candidate never parsed was reported `passed = true`. That rendered a session in which *every* stage was rejected as one in which every stage passed — the conflation FR-004 forbids, one level down, and the exact input an optimiser would learn nothing from. A stage with no verdict now reports `passed = null`: absence, not success.

7 The gate on building the loop

Do not build the optimiser until the corpus report shows the conformance measure VARIES across real sessions. If every recorded session scores identically, there is nothing for a loop to move, and the effort belongs on a different lever.

Even once it varies, the arithmetic is unforgiving, and it is arithmetic rather than pessimism. For a paired comparison, the exact floor at five tasks cannot reach significance under *any* outcome:

Distinct tasks	Best attainable p	Verdict
4	0.125	cannot reach $p < 0.05$
5	0.0625	cannot reach $p < 0.05$
6	0.03125	first size that can

Against a single-run pass@1 variance measured at **2.2–6.0 pp**, five blueprints support no inference at all: one task moves the pass rate 25 points. The power analysis is equally blunt: roughly **120–200 paired tasks** are needed to detect a 10 pp effect. The platform has five.

7.1 So the decision is: grow the corpus, or do not build the loop

In dependency order:

1. **Grow the held-out corpus.** Six is the arithmetic minimum; the power target is two orders of magnitude beyond it. This is the highest-value change and it is cheap.

2. **Run real sessions with the container runtime reachable**, so the diagnostic records accumulate and the variance question can actually be answered.
3. **Feed the reflector the rule histogram** instead of the build exit code.
4. **Build leave-one-out contribution**, then **eviction with a floor**.
5. **Only then** run the loop, scored on conformance.

8 Honest limits

Limit	Detail
Reflection's ceiling is unmeasured here	It captures $\approx 54\%$ of a trained curator's gain <i>on CODESKILL's benchmarks</i> . How much it recovers on repository-level Spring Boot code is not stated anywhere in the reports.
The direct repo-level test is null	SkillOpt on repository-level code measured +0.1 pp , not significant. This proposal is betting on conformance and cost, where the evidence is better, not on resolve rate.
The measure is conformance, not correctness	A well-layered but broken service scores full marks. A defect class nobody encoded reads clean.
Stage-local findings are blocked before persistence	The findings that survive into a saved artifact set come from whole-project rules only — a narrow signal, and its real-world variance is exactly what is still unmeasured.
Small n	Five blueprints. No significance is attainable. Repeated runs of one task are not independent observations and are never counted as a larger sample.
No trainable environment	No GPUs, no fine-tuning, no RL. The trained half of the CODESKILL result is out of reach by construction, not by choice.
The corpus currently holds no real records	The store held one record, and it was test output. A measurement store that cannot be told apart from its own fixtures is worthless; that is fixed, but it means the variance question is still open.

9 Decision record

Ref	Decision	Reason
D1	Optimise conformance, not build success	build success is binary and unmeasurable at $n = 5$
D2	The instrument is deterministic, not an LLM judge	a drifting judge makes a measured improvement unattributable
D3	Contribution by leave-one-out, per skill	the only training-free primitive that converts “it got better” into “this rule did it”
D4	Remove only below an evidence floor	“not proven to help” $\neq$ “hurts”; noise must not drive removals
D5	Reflect; never train	no compute for weight updates; reflection has a measured floor
D6	Normalise the measure for artifact-set size	a raw count rewards generating less code
D7	Attribution from the existing stage journal	it is already computed and free to keep
D8	Insufficiency is a reportable outcome	a point estimate without its power is noise reported as a result
D9	Demote the gate to a guardrail	it is a good regression veto and a bad acceptance criterion
D10	Bound the skills per round	attribution cost is skills $\times$ tasks

The one-sentence position. Text-space skill optimisation is real, bounded, and ranked last per unit of compute; its diagnostic channel is worth more than its loop, and the platform now has the channel, the record, and the report — so the next honest step is to grow the corpus and measure whether there is anything to optimise, not to build the optimiser.